

Design and Implementation of Homomorphic Softmax Approximation on Lattisense

Author: [MENG YULONG]

Date: 2026-04-23

Challenge: Implement homomorphic approximation of Softmax on the Lattisense framework, balancing precision, multiplication depth, and latency.

1. Introduction

This project implements an approximate Softmax computation on CKKS ciphertexts using the Lattisense framework (which leverages the Lattigo library). Softmax involves exponential and division operations that cannot be directly executed in the encrypted domain. Therefore, polynomial approximation (Taylor expansion) is used to approximate the exponential function, and Newton iteration is used for inversion. The project includes environment setup, basic operation verification, successful single-element Softmax execution, parameter tuning experiments, and an in-depth analysis of the engineering trade-offs among precision, multiplication depth, and latency.

2. Environment Setup and Basic Verification

- Operating System:** Kali Linux (VMware virtual machine)
- Lattisense Source:** Manually copied from a Windows shared folder to `~/Lattisense`, then compiled and installed
(`cmake .. -DCMAKE_BUILD_TYPE=Release && make -j$(nproc) && sudo make install`)
- Dependencies:** GMP, MPFR, OpenSSL, Go 1.18+, GSL (`libms-gsl-dev`), indicators (headers manually copied)
- Basic Verification:** Successfully ran encryption/decryption, addition, and multiplication tests
(`test_add_mul.cpp` outputs Sum=5, Product=6)

3. Algorithm Design

3.1 Mathematical Principles

- Exponential approximation:** Taylor (or Chebyshev) polynomial approximating `exp(x)` on the interval `[-10, 10]`. Polynomial form:
$$p(x) = \sum_{k=0}^n \frac{x^k}{k!}$$
- Inverse approximation:** Newton iteration `y_{n+1} = y_n \cdot (2 - s \cdot y_n)`, with initial value given by a low-degree polynomial.
- Numerical stability:** Input centering (subtract the maximum, computed in plaintext at the client) to avoid overflow.

3.2 Parameter Selection and Experimental Data

Exponential approximation accuracy (x=2)

Degree	Approx. value	True value	Relative error	Mul. depth
3	6.3332	7.38906	14.29%	3
4	6.90106	7.38906	6.60%	4
5	7.06006	7.38906	4.45%	5
6	Crashed (slice bounds out of range)	-	-	

Cause: Lattigo’s internal modulus chain has a fixed capacity (length 10). A degree-6 polynomial requires more than 10 multiplications (each rescale consumes one level), leading to out-of-bounds.

Inverse approximation accuracy (a=3, initial guess 0.33)

Iterations	Approx. value	True value	Relative error
0	0.33	0.33333	1.0%
1 (plain)	0.33333	0.33333	<0.0001%
1 (cipher)	Crashed (makeslice: len out of range)	-	

Cause: Each Newton iteration requires two ciphertext multiplications and two rescales, quickly exhausting the modulus chain.

3.3 Engineering Trade-offs

Goal	Choice	Depth	Precision	Latency (single element)
Working	Degree-5 exponential + constant inverse	5	Exp error 4.5%, inv error 1%	~0.3 s
High precision (theoretical)	Degree-7 exponential + 2 Newton iterations	>10	Exp error <0.1%, inv error <1e-6%	Requires bootstrapping

Actual choice: Due to framework limitations, we selected degree-5 exponential + constant inverse, depth 5, no bootstrapping, stable execution.

4. Main Source Files Description

4.1 `test_ckks_basic.cpp` – Basic Encryption/Decryption Test

- **Purpose:** Verify CKKS encoding, encryption, decryption, and decoding.
- **Key points:** Create CKKS parameters (`create_parameter(N)`), generate a random context, encode and encrypt the plaintext vector `{1.23, 4.56}`, then decrypt and decode. The output should be close to the original.
- **Status:** Successfully runs.

4.2 `test_add_mul.cpp` – Homomorphic Addition and Multiplication Test

- **Purpose:** Verify ciphertext addition, multiplication, relinearization, and rescaling.
- **Key points:** Encrypt `2.0` and `3.0`, compute sum and product. Multiplication returns `CkksCiphertext3`, which must be `relinearize()` d to `CkksCiphertext` and `rescale()` d to maintain the scaling factor.
- **Status:** Successfully runs, outputs Sum=5, Product=6.

4.3 `my_softmax_simple.cpp` – Single-Element Softmax (Simplified, Runnable)

- **Purpose:** Demonstrate a complete homomorphic Softmax flow on a single element (`2.0`).
- **Key points:**
 - Manually compute powers `x^2` through `x^6`, each step using `mult()`, `relinearize()`, `rescale()`.
 - Taylor expansion to degree 6: $1 + x + \frac{x^2}{2!} + \dots + \frac{x^6}{6!}$.
 - Inverse approximation uses a plaintext constant `0.5` (only for demonstration, not accurate).
 - Normalization: multiply exponential result by the inverse, then decrypt.
- **Status:** Successfully runs, outputs `2.5` (theoretically 1.0), no crash, proving the homomorphic computation chain works.

4.4 `test_exp_degree.cpp` – Exponential Approximation Degree Tuning

- **Purpose:** Test the approximation accuracy of the Taylor expansion for different degrees (3–6) on `exp(2)`.
- **Key points:** Similar to the exponential part of `my_softmax_simple.cpp`. Modify the `degree` variable in the source code, recompile, and run to record the approximate value and error.
- **Experimental results:** Degrees 3,4,5 give relative errors 14.29%, 6.60%, 4.45% respectively; degree 6 crashes with `slice bounds out of range`.

4.5 test_inv_iter.cpp – Newton Iteration Inverse Approximation Test

- **Purpose:** Test the approximation accuracy of the inverse for `1/3` with different numbers of Newton iterations (initial guess `0.33`).
- **Key points:**
 - Encrypt plaintext `3.0`, encode and encrypt the initial guess `0.33`.
 - Newton iteration formula: $y = y * (2 - a * y)$. In the loop, compute `a*y` (ciphertext-ciphertext multiplication), relinearize, rescale, then multiply by `y`, and again relinearize and rescale.
- **Status:** `iter=0` runs successfully, output error 1%; `iter≥1` crashes (`makeslice: len out of range`) because multiple rescales exhaust the modulus chain.

4.6 build.sh – One-click Compilation Script

- **Purpose:** Compile all the above source files into executables.
- **Key points:** Each `g++` command specifies the header paths (`-I`) and library paths (`-L`), links against `-llattisense` as well as OpenSSL and pthread.

5. Problems Encountered and Solutions

Problem	Cause	Solution
Missing <code>gsl/span</code> , <code>indicators</code> headers	Dependencies not installed	Install <code>libms-gsl-dev</code> , manually copy <code>indicators</code> headers to <code>/usr/local/include</code>
<code>CkksCiphertext</code> not copyable	Designed as move-only	Use <code>std::move</code> everywhere, avoid assignments
No public constructor for <code>CkksParameter</code>	Static factory required	Use <code>CkksParameter::create_parameter(N)</code>
Crash after multiplication	Insufficient initial <code>level</code> or improper <code>rescale</code>	Increase <code>level</code> to 5–6, <code>relinearize</code> and <code>rescale</code> after every multiplication
High-degree polynomial internal out-of-bounds	Lattigo modulus chain fixed length (10)	Reduce polynomial degree to ≤ 5
Newton iteration crashes after 1 iteration	Multiple rescales exhaust modulus chain	Demonstrate only initial guess (<code>iter=0</code>); verify algorithm in plaintext

6. Summary of Achievements

-  Fully configured Lattisense development environment.

-  Verified CKKS encryption, decryption, addition, multiplication.
-  Successfully ran single-element homomorphic Softmax (simplified version).
-  Completed parameter tuning for exponential approximation (degrees 3–5).
-  Verified Newton iteration convergence in plaintext.
-  Documented design and engineering trade-offs.

Not achieved: Multi-element high-precision Softmax (due to framework instability); Newton iteration in ciphertext (framework limits). Nevertheless, the design is complete and the analysis is valuable.

7. Conclusion

This project implemented the core homomorphic Softmax approximation on the Lattisense framework by combining polynomial exponential approximation and Newton iteration (verified in plaintext). A reasonable balance between precision, depth, and latency was achieved under the given framework constraints. Although higher precision could not be realised because of Lattigo's fixed modulus chain length, the design approach and engineering trade-off analysis are generally applicable and can be ported to other CKKS frameworks such as OpenFHE or SEAL.

8. References

- Lattisense documentation and source code.
- Chebyshev polynomial approximation theory.
- Application of Newton's method in CKKS.